Lecture 9 – Coding Exercise: Animal System

Topics: Abstract classes, multiple interfaces, inheritance, constructors, method overriding, runtime polymorphism, arrays, `instanceof`, and downcasting.

Part 1: Create an Abstract Class `Animal`

Create an abstract class named `Animal`.

Requirements:

1. Declare the following protected instance variables:
    - `name` (String)
    - `age` (int)
2. Create a constructor to initialize both variables.
3. Create the following methods:
    - `displayInfo()` – prints the animal's name and age.
    - `makeSound()` – an abstract method.
4. Declare a static variable `animalCount` to count the number of animals created.
5. Increase `animalCount` by 1 whenever an Animal object is created.

Part 2: Create Two Interfaces

Interface 1: `Pet`

Declare the following methods:

- `play()` – prints a message indicating that the animal is playing.
- `eat()` – prints a message indicating that the animal is eating.

Interface 2: `Trainable`

Declare the following method:

- `performTrick()` – prints a message indicating that the animal is performing a trick.

Part 3: Create a `Cat` Class

Requirements:

1. Extend the `Animal` class.
2. Implement both `Pet` and `Trainable` interfaces.
3. Declare an instance variable:
    - `color` (String)

4. Create a constructor to initialize `name`, `age`, and `color`.
5. Use `super()` to initialize the superclass variables.
6. Override the following methods:
    - `makeSound()` – prints `"Meow!"`
    - `play()` – prints that the cat is playing.
    - `eat()` – prints that the cat is eating.
    - `performTrick()` – prints that the cat is performing a trick.

Part 4: Create a `Dog` Class

Requirements:

1. Extend the `Animal` class.
2. Implement both `Pet` and `Trainable` interfaces.
3. Declare an instance variable:
    - `breed` (String)
4. Create a constructor to initialize `name`, `age`, and `breed`.
5. Use `super()` to initialize the superclass variables.
6. Override the following methods:
    - `makeSound()` – prints `"Woof!"`
    - `play()` – prints that the dog is playing.
    - `eat()` – prints that the dog is eating.
    - `performTrick()` – prints that the dog is performing a trick.

Part 5: Create the `Main` Class

Complete the following tasks inside `main()`.

A. Create Objects

Create the following objects:

- Cat: Kitty, age 2, color White
- Cat: Mimi, age 3, color Black
- Dog: Buddy, age 4, breed Golden Retriever
- Dog: Max, age 5, breed German Shepherd

B. Create an Array

1. Create an `Animal` array named `animals`.
2. Store all four objects in the array.
3. Use a `for` loop to traverse the array.
4. Call `displayInfo()` and `makeSound()` for each animal.

C. Use `instanceof` and Downcasting

Traverse the `animals` array.

1. Check whether each animal is a `Cat` or a `Dog` using `instanceof`.
2. Downcast the object to its actual class.
3. Display the additional information:
    - Cat: color

- Dog: breed

D. Use Interface References

Create the following references:

1. A `Pet` reference pointing to a `Cat` object.
2. A `Pet` reference pointing to a `Dog` object.
3. A `Trainable` reference pointing to a `Cat` object.
4. A `Trainable` reference pointing to a `Dog` object.

Call the appropriate methods using these references.

E. Use an Interface Array

1. Create a `Trainable` array named `trainableAnimals`.
2. Store the Cat and Dog objects in the array.
3. Use a `for` loop to call `performTrick()` for every object.

F. Display the Total Number of Animals

Display the total number of animals created using `animalCount`.

Sample Output

```
========== Animal Information ==========

Name: Kitty
Age: 2
Meow!

Name: Mimi
Age: 3
Meow!

Name: Buddy
Age: 4
Woof!

Name: Max
Age: 5
Woof!

========== Animal Details ==========

Cat: Kitty
Color: White

Cat: Mimi
Color: Black

Dog: Buddy
Breed: Golden Retriever

Dog: Max
Breed: German Shepherd

========== Pet Activities ==========
```

```
Kitty is playing.
Kitty is eating.

Buddy is playing.
Buddy is eating.

========== Training Activities ==========

Kitty is performing a trick.
Mimi is performing a trick.
Buddy is performing a trick.
Max is performing a trick.

========== Total Animals ==========

Total animals: 4
```

Main.java

```
public class Main {
    public static void main(String[] args) {

        // 1. Create Objects
        Cat c1 = new Cat("Kitty", 2, "White");
        Cat c2 = new Cat("Mimi", 3, "Black");

        Dog d1 = new Dog("Buddy", 4, "Golden Retriever");
        Dog d2 = new Dog("Max", 5, "German Shepherd");


        // 2. Create an Animal Array
        // TODO: Store all four objects in an Animal array named animals.


        // 3. Display Animal Information
        System.out.println("========== Animal Information ==========");

        // TODO: Use a for loop to traverse the animals array.
        // Call displayInfo() and makeSound() for each animal.


        // 4. Use instanceof and Downcasting
        System.out.println("========== Animal Details ==========");

        // TODO: Use instanceof to check whether each object is a Cat or Dog.
        // Downcast the object and display its additional information.
        // Cat: display color
        // Dog: display breed


        // 5. Use Pet Interface References
        System.out.println("========== Pet Activities ==========");

        // TODO: Create two Pet references:
        // One pointing to c1 and one pointing to d1.
        // Call play() and eat() using these references.


        // 6. Use Trainable Interface References
        System.out.println("========== Training Activities ==========");

        // TODO: Create two Trainable references:
        // One pointing to c1 and one pointing to d1.
        // Call performTrick() using these references.
```

```
        // 7. Create a Trainable Array
        // TODO: Create a Trainable array named trainableAnimals.
        // Store all four objects in the array.
        // Use a for loop to call performTrick() for each object.


        // 8. Display Total Animals
        System.out.println("========== Total Animals ==========");

        // TODO: Display the static variable animalCount from Animal.

    }
}
```

Additional Questions

1. Can a class extend one abstract class and implement multiple interfaces?
2. What is the difference between `extends` and `implements`?
3. Why can both `Cat` and `Dog` be stored in an `Animal[]` array?
4. Why can both `Cat` and `Dog` be stored in a `Trainable[]` array?
5. Can we call `performTrick()` using an `Animal` reference? Explain.
6. Can an interface reference point to an object of a class that implements that interface?
7. What happens if a class implements an interface but does not implement all its abstract methods?